Module : Introduction au Machine Learning

TP Général

Partie 1 : Pre-processing (Nettoyage et Transformation)

Dataset : loan_prediction.csv (Risque de Crédit)
Contexte : Une banque souhaite automatiser le processus d'éligibilité aux prêts (fictif).

Étape 1.1 : Compréhension des Données

1. **Importation :** Importez pandas, numpy, matplotlib, seaborn.
2. **Chargement :** Chargez le fichier loan_prediction.csv dans un DataFrame.
3. **Inspection Initiale :**
   - Affichez les 5 premières lignes (head()).
   - Utilisez info() pour repérer les types (float, object) et les valeurs nulles.
   - Utilisez describe() pour voir la distribution des colonnes numériques (ApplicantIncome, LoanAmount, etc.).
4. **Doublons :** Vérifiez et supprimez les éventuelles lignes dupliquées.
5. **Visualisation :**
   - Créez un histogramme pour la colonne ApplicantIncome (Revenu du demandeur).
   - Créez une heatmap de corrélation pour visualiser les liens entre les variables numériques.

Étape 1.2 : Nettoyage des Données

1. **Valeurs Manquantes :**
   - Identifiez les colonnes contenant des NaN (ex: Gender, Married, LoanAmount, Credit_History).
   - **Stratégie 1 (Numérique) :** Imputez les valeurs manquantes de LoanAmount avec la **moyenne**.
   - **Stratégie 2 (Catégorielle) :** Imputez les valeurs manquantes de Gender et Self_Employed avec le **mode** (la valeur la plus fréquente).
   - **Stratégie 3 (Critique) :** Supprimez les lignes où Credit_History est manquant (car c'est une variable trop critique pour être devinée).
2. **Valeurs Aberrantes (Outliers) :**
   - Analysez la variable ApplicantIncome avec un Boxplot.
   - Calculez l'écart interquartile (IQR).
   - Filtrez les données pour supprimer les revenus extrêmes (au-delà de $Q3 + 1.5 \times IQR$).

Étape 1.3 : Transformation

1. **Sélection de features :** Supprimez la colonne identifiant Loan_ID (inutile pour le modèle).
2. **Encodage :**
   - Transformez la variable Property_Area (Urban, Semiurban, Rural) avec un **One-Hot Encoding**.
   - Transformez la variable cible Loan_Status (Y/N) en binaire (1/0) avec LabelEncoder ou map.

3. **Mise à l'échelle :**
   - Appliquez une **Standardisation** (StandardScaler) sur les variables ApplicantIncome et LoanAmount.

Partie 2 : Supervised Learning - Classification

Dataset : Wine (Analyse chimique de vins)
Source : Scikit-Learn Datasets (178 échantillons, 13 caractéristiques, 3 classes de vin)

Étape 2.1 : Préparation

1. **Chargement :**
   Python
   ```python
   from sklearn.datasets import load_wine
   wine = load_wine()
   X = wine.data
   y = wine.target
   ```

   Affichez les noms des features (wine.feature_names) et des classes (wine.target_names).
2. **Séparation :** Divisez les données en Train (80%) et Test (20%) avec random_state=42.

Étape 2.2 : Modélisation et Comparaison

Entraînez ces trois modèles sur X_train et y_train :

Modèle A : KNN (K-Nearest Neighbors)

1. Importez et instanciez KNeighborsClassifier avec $K=5$.
2. Entraînez le modèle.
3. Affichez l'Accuracy score sur le jeu de test.

Modèle B : Random Forest (Forêts Aléatoires)

1. Importez RandomForestClassifier.
2. Instanciez le modèle avec 100 arbres (n_estimators=100).
3. Entraînez et affichez la précision.
   - *Question bonus :* Affichez l'importance des variables (feature_importances_) pour voir quel composant chimique influence le plus la classe du vin.

Modèle C : SVM (Support Vector Machine)

1. Importez SVC.
2. Entraînez un modèle avec un noyau kernel='rbf' (souvent meilleur pour ce dataset que 'linear').
3. Affichez l'Accuracy.

Étape 2.3 : Évaluation Avancée (Focus sur le meilleur modèle)

Choisissez le modèle ayant obtenu le meilleur score (souvent Random Forest ou SVM sur ce dataset).

1. **Matrice de Confusion :** Affichez-la avec seaborn.heatmap. Observez s'il y a des confusions entre la classe 1 et la classe 2.
2. **Rapport complet :** Affichez le classification_report.
3. **Cross-Validation :** Vérifiez la robustesse du modèle avec une validation croisée à 5 plis (cv=5).

Partie 3 : Supervised Learning - Régression Linéaire Simple

Dataset : California Housing (Extrait)
Source : Scikit-Learn Datasets
Objectif : Prédire la valeur médiane d'une maison (MedHouseVal) uniquement en fonction du revenu médian du quartier (MedInc).

1. **Chargement et Sélection :**
   Python
   ```python
   from sklearn.datasets import fetch_california_housing
   housing = fetch_california_housing()
   df = pd.DataFrame(housing.data, columns=housing.feature_names)
   df['Target'] = housing.target

   # On ne garde que deux colonnes pour la régression simple
   X_reg = df[['MedInc']]  # Revenu Médian
   y_reg = df['Target']    # Valeur de la maison (en 100k$)
   ```
2. **Séparation :** Divisez en Train/Test.
3. **Entraînement :** Entraînez une LinearRegression.
4. **Visualisation :**
   - Affichez le nuage de points (scatter) des données de test (en bleu).
   - Tracez la droite de régression prédite par le modèle (en rouge).
5. **Prédiction :**
   - Si le revenu médian d'un quartier est de **5.0** (soit environ 50 000$), quel est le prix prédit pour une maison ?

Partie : Apprentissage Non Supervisé

1. Contexte

Vous êtes Data Engineer chez l'usine **"Tech-Industries"**. Le directeur de production vous confie trois missions basées sur les données des capteurs de l'usine :

1. **Regroupement de pièces (Clustering)** : Identifier des familles de pièces usinées selon leurs dimensions.
2. **Analyse de signaux (DBSCAN)** : Analyser des zones de couverture wifi dans l'entrepôt (données non linéaires).
3. **Reporting (PCA)** : Visualiser l'état de santé des machines (30 capteurs) sur un écran de contrôle 2D.
4. **Maintenance Prédictive (Anomalies)** : Détecter les pièces défectueuses avant l'expédition.

2. Préparation de l'environnement

Copiez ce code pour importer les modules nécessaires. Notez l'ajout de make_circles et load_breast_cancer.

Python

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Modules Scikit-Learn
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.ensemble import IsolationForest
from sklearn.datasets import make_blobs, make_circles, load_breast_cancer
from sklearn.preprocessing import StandardScaler

# Configuration visuelle
plt.style.use('seaborn-v0_8') # ou 'ggplot'
np.random.seed(42)
print("Environnement prêt.")
```

---

3. Partie 1 : Segmentation des Pièces (K-Means)

L'usine produit des pièces métalliques. Nous avons extrait deux mesures principales : le *Poids* et la *Résistance*.

Tâche 1.1 : Génération et Visualisation

Python

```
# Génération de 400 pièces réparties en 3 groupes naturels
X_pieces, _ = make_blobs(n_samples=400, centers=3, cluster_std=0.8, random_state=42)

# Visualisation
plt.figure(figsize=(8, 6))
plt.scatter(X_pieces[:, 0], X_pieces[:, 1], s=30, alpha=0.7)
plt.title("Mesures des Pièces (Non Étiquetées)")
plt.xlabel("Poids (standardisé)")
plt.ylabel("Résistance (standardisé)")
plt.show()
```

Tâche 1.2 : La Méthode du Coude (Elbow Method)

Nous voulons vérifier s'il existe bien 3 types de pièces ou plus.

- Utilisez une boucle pour tester **K** allant de 1 à 10.
- Stockez l'inertie (kmeans.inertia_) pour chaque **K**.
- Tracez la courbe. Confirmez-vous que le "coude" est à **K=3** ?

Tâche 1.3 : Application de K-Means

- Entraînez KMeans avec **K=3**
- Visualisez les clusters avec des couleurs distinctes.
- Affichez les **centroïdes** (le "prototype" de chaque pièce) en rouge.

---

4. Partie 2 : Géométries Complexes (K-Means vs DBSCAN)

Les robots de transport circulent dans l'entrepôt selon des trajectoires circulaires concentriques. K-Means est-il capable de comprendre cette géométrie ?

Tâche 2.1 : Échec de K-Means sur des cercles

Python

```
# Génération de données en cercles concentriques
X_circles, _ = make_circles(n_samples=400, factor=0.5, noise=0.05, random_state=42)

# Application de K-Means (k=2 car nous avons deux cercles)
kmeans_circles = KMeans(n_clusters=2, random_state=42)
y_pred_km = kmeans_circles.fit_predict(X_circles)

# Visualisation
plt.scatter(X_circles[:, 0], X_circles[:, 1], c=y_pred_km, cmap='viridis')
plt.title("Résultat K-Means sur Cercles")
plt.show()
```

- **Question :** Pourquoi le résultat est-il incohérent géométriquement ?

Tâche 2.2 : La Solution DBSCAN

- Utilisez l'algorithme DBSCAN.
- Paramètres suggérés : eps=0.15, min_samples=5.
- Affichez le résultat. DBSCAN arrive-t-il à séparer le petit cercle intérieur du grand cercle extérieur ?

---

5. Partie 3 : Réduction de Dimension (ACP/PCA)

Chaque machine de l'usine possède 30 capteurs de diagnostic (vibrations, température, pression, etc.). Il est impossible pour un humain de lire 30 graphiques en même temps. Nous allons utiliser le dataset *Breast Cancer* (30 features) pour simuler ces données de capteurs.

Tâche 3.1 : Chargement et Standardisation

Python

```
# Chargement des données (simulant les capteurs machines)
data_sensors = load_breast_cancer()
X_sensors = data_sensors.data
print(f"Dimensions originales : {X_sensors.shape}") # Devrait être (569, 30)

# Standardisation (OBLIGATOIRE pour PCA)
scaler = StandardScaler()
X_sensors_scaled = scaler.fit_transform(X_sensors)
```

Tâche 3.2 : Projection 2D

- Utilisez PCA pour réduire les données de **30 dimensions vers 2 composantes principales**.
- Affichez le nuage de points résultant.
- Colorez les points selon la cible (c=data_sensors.target) pour voir si les états "Sain" (Bénin) et "Critique" (Malin) se séparent bien visuellement, même après avoir perdu 28 dimensions.
- Analysez pca.explained_variance_ratio_ : Quel pourcentage d'information contient votre graphique 2D ?

---

6. Partie 4 : Détection d'Anomalies (Isolation Forest)

Le système de sécurité doit détecter des intrusions réseau ou des pannes capteurs rares.

Tâche 4.1 : Simulation

Python

```
# Données normales (Fonctionnement standard)
X_regular, _ = make_blobs(n_samples=300, centers=1, cluster_std=1.0, random_state=42)
# Anomalies (Pannes rares / Intrusions : très dispersées)
X_outliers, _ = make_blobs(n_samples=15, centers=1, cluster_std=10.0, center_box=(-10, 10),
random_state=42)
# Fusion
X_system = np.vstack([X_regular, X_outliers])
```

Tâche 4.2 : Isolation Forest

- Entraînez un modèle IsolationForest.
- Réglez le paramètre contamination=0.05 (nous estimons qu'il y a environ 5% de pannes).
- Utilisez fit_predict() pour détecter les anomalies (les valeurs -1).
- Visualisez le résultat : Affichez les points normaux en bleu et les anomalies détectées en rouge. Le modèle a-t-il correctement "isolé" les points éloignés ?